



FEYZİYE SCHOOLS FOUNDATION

IŞIK UNIVERSITY

IŞIK UNIVERSITY

Faculty of Economics, Administrative, and Social Sciences

Department of Information Technologies

B. S. Thesis

Mushroom Identification System

by

Mehmet KEPUR

21MISY1007

Supervised by: Dr. *Gulsum Cigdem CAVDAROGLU*

06-2025

Mushroom Identification System

A Project Presented

by

Mehmet KEPUR

21MISY1007

to

The Department of Information Technologies

in partial fulfillment of the requirements

for the degree of

Bachelor of Science

in the field of

Management Information Systems

Işık University

İstanbul, Turkey

06-2025

© Copyright 2025 by Mehmet KEPUR

All Rights Reserved

Abstract

In this study, a mushroom classification system was developed using Convolutional Neural Networks (CNNs) to identify different mushroom species based on image data. The project aimed to support educational and research purposes by enabling users to classify mushrooms through a user-friendly mobile application. A custom dataset containing 26 mushroom species and a total of 2600 images was used. The images were preprocessed and resized to 375×500 pixels, and data augmentation techniques such as rotation, zooming, and flipping were applied to increase model stability.

The classification model was built using the ResNet50 architecture with transfer learning in TensorFlow. Despite some signs of overfitting, the model achieved a training accuracy of 100% and a validation accuracy of approximately 79% after 20 epochs. To improve accessibility and usability, a mobile application was developed using Flutter, allowing users to upload images directly from their device. The app communicates with a Flask-based API, which handles image preprocessing and model inference on the backend.

To ensure consistent and reliable results, the system currently supports only .jpg and .jpeg image formats, minimizing preprocessing errors. While the system does determine edibility, it doesn't recommend consuming wild mushrooms through the application. The app provides valuable insights into species identification and family grouping. Future improvements include enhancing the questioning logic with decision trees and bigger database to increase model reliability and supporting user interaction for community-based learning.

This work demonstrates the potential of deep learning and mobile technologies in supporting biological classification tasks and encourages further development in biodiversity-related applications.

Acknowledgements

I would like to express my gratitude to my advisor, Dr. Gülsüm Çiğdem Çavdaroğlu Akkoç, from the Faculty of Economics and Administrative and Social Sciences at Işık University, for her continuous support and guidance throughout the course of this project. Her insightful feedback and expertise were invaluable to the development and completion of this work.

I would also like to sincerely thank my friends who took the time to test the application and provided valuable feedback. Their contributions helped me improve both the functionality and usability of the system, and their support motivated me through the challenges of this project.

Contents

Abstract	2
Acknowledgements	3
Contents	4
List of Figures	6
Chapter 1 Introduction	7
1.1 Overview	7
1.2 Motivation	8
1.3 Contributions	9
1.4 Outline of the Project	10
Chapter 2 Literature Review	12
Chapter 3 Materials and Methods	17
3.1 Overview	17
3.2 Dataset Description	17
3.3 JupyterLab and Python	18
3.4 CNN Model Architecture	18
3.4.1 Keras	19
3.4.2 TensorFlow	19
3.4.3 NumPy	20
3.4.4 ResNet50	20
3.5 Flask	21
3.6 Flutter	21
Chapter 4 Implementation of the Project	23
4.1 Overwiev	23
Chapter 5 Results and Discussion	31
5.1 Overview	31
5.2Results	31
5.3 Discussion	33

Chapter 6 Conclusion	35
References.....	37

List of Figures

4.1 1 Main Page.....	24
4.1 2 Warning.....	25
4.1 3 Prediction Page.....	26
4.1 4 Information Page	27
4.1 5 Report Section	28
4.1 6 Low Confidence Warning	29
4.1 7 Prediction History	30
5.2 1 Accuracy-Loss Plot	32
5.2 2 Overfitting	32

Chapter 1 Introduction

1.1 Overview

Mushrooms, as a diverse group of fungi, play a critical role in ecosystems and are widely appreciated for their culinary, medicinal, and ecological importance. It is known that there are approximately 2200 macro fungus species in Türkiye. Approximately 300 of them edible natural mushroom species [1]. However, their diversity also introduces significant challenges, especially in species identification. There has been minimal research on whether they are edible or dangerous. For example, *Amanita phalloides* is the most dangerous, poisonous mushroom species in our climatic conditions. It is the cause of 90-95% of all deaths due to mushroom poisoning [2]. Learning machine will be able to classify if a mushroom is poisonous or not using data mining as one of the approaches for obtaining computer-assisted knowledge [3]. Accurate classification of mushrooms is not only essential for scientific and environmental purposes but also crucial for public safety, given the presence of many toxic varieties that resemble edible ones.

In recent years, the integration of artificial intelligence with image processing has opened new possibilities in the field of species recognition. Convolutional Neural Networks (CNNs), in particular, have shown remarkable success in various classification tasks due to their ability to learn visual patterns from data. This project leverages the power of CNNs—specifically, a fine-tuned ResNet50 model—to classify mushrooms based on visual input and provide family group identification. Previous studies have shown that CNN-based models, particularly those utilizing the ResNet50 architecture with Transfer Learning, have been successfully applied to image classification tasks such as plant disease identification. This demonstrates the versatility and effectiveness of similar approaches in various domains [4].

Beyond building a machine learning model, this study also emphasizes usability and accessibility. A mobile application was developed using Flutter, enabling users to upload images and receive classification results through a backend Flask API. This practical implementation is designed to assist mushroom enthusiasts, students, and researchers in making fast, informed observations about mushroom species. The app does not aim to identify edibility for safety reasons, but rather supports educational engagement with mycology and biodiversity.

By combining deep learning, mobile development, and user-centered design, the project demonstrates a modern approach to biological classification challenges. It also highlights how technology can bridge the gap between scientific tools and everyday users in a responsible and informative manner.

1.2 Motivation

Mushrooms are biologically diverse and ecologically significant organisms that play an essential role in various ecosystems. However, due to their vast variety and close visual resemblance between species, distinguishing between edible and toxic mushrooms remains a challenge. Mushroom poisoning, which can lead to severe health complications or even death, continues to be a recurring issue worldwide. This highlights a growing need for accessible and reliable identification tools to support awareness and safety.

In parallel, advances in computer vision and artificial intelligence have enabled the development of powerful image classification systems. Convolutional Neural Networks (CNNs), in particular, have demonstrated significant success in visual recognition tasks. Leveraging these technologies to assist in species identification presents a valuable opportunity to minimize risks associated with mushroom misidentification.

This project is motivated by the potential to contribute to public education and awareness, while also exploring how AI-based solutions can be applied in nature-related fields. Additionally, the project serves as a practical implementation of deep learning models, providing a platform to study and enhance neural network performance in real-world applications. The underlying methods developed for mushroom classification can also be adapted to other domains such as agriculture, biology, and environmental monitoring, where accurate image-based classification is essential.

By building a mobile-friendly system supported by a trained CNN model, this project aims to increase accessibility to species-level identification, foster responsible interaction with nature, and demonstrate the broader applicability of artificial intelligence in societal and ecological contexts.

1.3 Contributions

This study offers both technical and practical contributions in the field of computer vision and applied machine learning, with a focus on biological image classification. The key contributions of the project are as follows:

Technical Contributions

- **Development of a CNN-Based Classification Model:** A Convolutional Neural Network (CNN) architecture based on ResNet50 was implemented to classify mushroom species. The model was trained on a multi-class image dataset consisting of 31 mushroom species, achieving approximately 79% validation accuracy.
- **Data Processing and Augmentation Pipeline:** A comprehensive preprocessing pipeline was established to handle class imbalance and dataset limitations. Techniques such as data augmentation (e.g., rotation, zoom, flipping) were applied to enhance model robustness.
- **Cross-Platform Deployment via API:** A Flask-based backend API was developed to serve the trained model and handle image classification requests. This API enables integration with various client applications, making the system modular and extensible.
- **Mobile Application Interface:** A Flutter-based mobile application was designed to provide users with a simple and accessible interface to upload mushroom images and receive classification results in real time.

Practical Contributions

- **Support for Public Safety and Awareness:** By enabling fast and accessible identification of mushroom species, the application supports efforts to reduce incidents related to mushroom poisoning and increases general awareness of mushroom diversity.
- **Educational Value:** The system may be used as an educational aid for biology students, nature enthusiasts, or amateur mycologists seeking to learn more about mushroom species and taxonomy.
- **Foundation for Broader Applications:** The methodologies used in this project can be adapted for other domains that require visual recognition of natural objects, such as plant disease detection, leaf classification, or agricultural monitoring.

Scientific Contribution

- **Application of Deep Learning in Biodiversity Studies:** This project contributes to the growing body of research that applies artificial intelligence to biodiversity and environmental science. It demonstrates how modern machine learning techniques can assist in solving real-world classification problems involving complex natural data.

1.4 Outline of the Project

This thesis is structured into five main chapters, each addressing a distinct aspect of the mushroom classification system:

Chapter 1 – Introduction:

Introduces the background, motivation, objectives, and contributions of the project. It also outlines the significance of applying artificial intelligence in biological image classification tasks such as mushroom recognition.

Chapter 2 – Literature Review:

Explores existing studies and approaches related to mushroom classification, deep learning applications in biodiversity, and relevant work in convolutional neural networks. This chapter helps to contextualize the current project within the broader research landscape.

Chapter 3 – Materials and Methods:

Details of the technical approach adopted in the project, including dataset collection and preparation, preprocessing techniques, model architecture (ResNet50), training strategies, and tools used (e.g., TensorFlow, Flask, Flutter). It also explains how the system was designed to handle image classification and user interaction.

Chapter 4 – Implementation of the Project:

Discusses the practical aspects of the project's development, including model training, handling overfitting, evaluation setup, and deployment steps. This chapter also covers the creation of the mobile application and its integration with the backend.

Chapter 5 – Results and Discussion:

Presents the outcomes of the model training and testing processes. Key performance metrics are analyzed, and the effectiveness of the system is evaluated. The strengths and limitations of the approach are discussed, along with potential real-world applications and improvements.

Chapter 6 – Conclusion:

Summarizes the project by highlighting the development of a CNN-based mushroom classification system integrated into a mobile app. Mentions the high training accuracy, user-friendly design, and the educational purpose of the app. Finally, emphasizes the potential for future improvements and its contribution to deep learning applications in mushroom identification.

Chapter 2 Literature Review

Several studies focus on leveraging machine learning and image processing techniques to identify edible and poisonous mushrooms. For instance, a study conducted in Sri Lanka utilized Convolutional Neural Networks (CNNs) to classify mushrooms based on physical attributes like shape, size, and color. They achieved a 90% accuracy rate for edible mushroom identification and noted the potential for further improvement through feature optimization and user feedback integration. Studies emphasize the importance of a robust and diverse dataset. High-resolution images captured under consistent lighting conditions significantly improved model accuracy. Preprocessing methods, such as resizing, normalization, and data augmentation (e.g., flipping and rotating images), helped models generalize better to unseen data [5].

The cultivation of edible mushrooms is a trend in the Sri Lankan market. Unfortunately, certain types of mushrooms are deadly and contain toxins that can make people sick or even lead to death. Around 70% of all-natural poisoning is caused by mushrooms, and it frequently results in death. But nonetheless, out of the hundreds of species found on Earth, only 30 to 50 are toxic [6].

The approach will use advanced image processing techniques, such as feature extraction, segmentation, and classification, to analyze images of mushrooms and distinguish between edible and non-edible mushrooms and edible mushroom species. This method has the potential to provide a more accurate and efficient way of identifying mushrooms, which will improve the safety and accessibility of wild mushroom harvesting, promote the consumption of nutritious edible mushrooms, and prevent accidental ingestion of poisonous mushrooms [7].

The primary challenges include limited dataset and complex classification requirements such as color, shape or even habitat (Bandara et al. 2023).

There are alternatives for data sourcing that we can identify datasets from public repositories like Kaggle, MushroomObserver or iNaturalist. These repositories will be extremely helpful in building a machine learning model and training our predefined database.

MushroomObserver, an open-source database, has been extensively used for gathering species-specific data, including high-resolution images and metadata, to train classification models. The platform supports biodiversity studies by offering a rich collection of mushroom observations contributed by users globally (Smith et al., 2020).

Datasets from Kaggle have been widely employed in AI-based classification tasks. The Mushroom Classification dataset, available on Kaggle, includes detailed features such as cap shape, color, and odor, enabling researchers to benchmark models for accurate edible and poisonous mushroom identification (Jones et al., 2018).

The iNaturalist species classification and detection dataset was developed to reflect the complexities of real-world biodiversity, unlike many standard image classification datasets that have a balanced distribution of categories. It contains over 859,000 images representing more than 5,000 plant and animal species. The dataset presents significant challenges such as visually similar species, varying image quality, and class imbalance. These images were collected from diverse environments using different camera types and were validated by multiple citizen scientists, making it highly suitable for training robust models in real-world scenarios (Van Horn et al., 2018).

Over the past decade, deep neural networks—especially convolutional neural networks (CNNs), have revolutionized computer vision; however, these models typically require large amounts of labeled data to achieve high accuracy. In practical scenarios, datasets are often small or imbalanced, leading to overfitting, where models perform well on training data but struggle with unseen inputs [8].

Preprocessing data such as resizing and normalization to fit the input dimensions of the AI model is one of the important previous step before we decide to build computer vision model such as mushroom identification system.

Image preprocessing steps, including resizing and normalization, are essential for ensuring consistency across input data. For instance, resizing all images to a fixed size of 224x224 pixels was a standard preprocessing step in training CNNs, ensuring compatibility with the model's architecture (Simonyan & Zisserman, 2015). Normalization, by scaling pixel values to a range between 0 and 1, has been shown to improve convergence during training by standardizing input distributions (Krizhevsky et al., 2012).

The one of the algorithms for classification one of those study are logistic regression, k-nearest neighbor (KNN), and XGBoost. Three methods were chosen to obtain a multiple comparison. Starting with logistic regression, an uncomplicated regression analysis provides the probability of an event occurring, ideal for outputting a binary target variable. KNN is a “lazy learning” classification model that uses proximity to make classifications or predictions

about the clustering of an individual data point. XGBoost is an already optimized method for classification and regression, ideal for large data sets or data sets that present a mixture of categorical and numerical varieties and simple to use [9]. KNN could be useful for classification projects because it's easy to implement and there is no model training required because it uses training data directly. The cons of this method could be being slow on prediction time since it computes distances to all points in the dataset. Therefore, there might be performance drop with large datasets.

Artificial neural networks (ANN) are parallel computational models with highly linked adaptive processing units that can learn from experience as well as acquire new information. One use of ANN is the classification of mushrooms as either edible or poisonous [10].

Various classification algorithms, including K-Nearest Neighbors (KNN), Naive Bayes, and Decision Trees, have been explored:

KNN demonstrated high performance in classifying mushroom datasets with minimal preprocessing but required significant computational resources for large datasets

Random Forests and Decision Trees excelled in accurately distinguishing edible and poisonous species using hierarchical decision structures. The C4.5 algorithm, a variant of Decision Trees, was noted for its efficiency in reducing classification errors [11].

Designing an effective CNN architecture involves selecting the number of convolutional layers, kernel sizes, and activation functions based on the dataset characteristics and task requirements. For example, architectures like AlexNet and VGG16 demonstrated the significance of deep hierarchical layers for feature extraction, with smaller kernel sizes providing better generalization (Krizhevsky et al., 2012; Simonyan & Zisserman, 2015). Custom architectures are often tailored by experimenting with hyperparameters and utilizing transfer learning for faster convergence (Howard et al., 2017).

Model training involves feeding the processed dataset into the CNN and optimizing the network's parameters using algorithms like Stochastic Gradient Descent (SGD) or Adam. The dataset is typically divided into training, validation, and test sets to ensure generalization and prevent overfitting. For example, researchers trained a ResNet model on a processed image dataset with a split of 70% for training and 30% for validation and achieved state-of-the-art accuracy for image classification tasks (He et al., 2016).

In one of the computer vision project done by M. Tan et al., they experimented with EfficientNetV2 models that train significantly faster than state-of-the-art models while being up to 6.8 times smaller. Training speed can be further improved by gradually increasing the image size during training, though this often leads to a drop in accuracy. To mitigate this issue, they introduce an enhanced progressive learning approach that adaptively adjusts regularization techniques, such as data augmentation, alongside image size. With this method, EfficientNetV2 achieves superior performance compared to previous models on the ImageNet, CIFAR, Cars, and Flowers datasets (M. Tan et al, 2021).

In their investigation, Nusrat, Zahid, and the other researchers concentrated on identifying different species of mushrooms. They used deep learning methods such as InceptionV3, VGG16, and Resnet50 to classify 8190 images of mushrooms [12]. They decided to divide the training and test data by 8:2. To obtain the highest test accuracy, they used InceptionV3 and the Contrast Limited Adaptive Histogram Equalization (CLAHE) method to compare contrast-enhanced and non-contrast-enhanced approaches. With an accuracy of 88.40%, InceptionV3 has the highest success rate when compared to other algorithms. In their study, Wacharaphol et al. tackled the problem of distinguishing between harmful and edible mushrooms because of their similar appearances [13]. Using convolutional neural networks (CNN) and region convolutional neural networks (R-CNN), they categorized the edible and toxic forms of five mushroom species that are frequently found in Thailand: *Inocybe rimosa*, *Amanita phalloides*, *Amanita citrina*, *Russula delica*, and *Phaeogyroporus portentosus*. The goal of the study was to reduce the number of deaths caused by toxic mushroom consumption. The study compared the accuracy and test time of three models. Their proposed model had accuracy scores of 95.50% and required less time for training and testing than other CNN studies for mushroom classification [14].

A study was conducted to evaluate the accuracy of three different pre-trained Convolutional Neural Network (CNN) models for image classification. The objective was to determine which model offered the highest recognition accuracy. The dataset used for this research was sourced from ImageNet, with a focus on image recognition, dataset size, classification techniques, and recognition accuracy.

The three implemented machine learning models in this study were VGG-16, InceptionV3, and EfficientNetB7. The models achieved the following accuracies:

- VGG-16: 97.67%

- InceptionV3: 97.2%
- EfficientNetB7: 99%

Based on the results, EfficientNetB7 outperformed the other two models, demonstrating the highest accuracy at 99%. This indicates that EfficientNetB7 is the most effective model among the three for image classification tasks in this study [15].

Expanding datasets with continuous data through active learning and user-generated inputs has been shown to improve model accuracy and robustness. For instance, a study on real-time object detection employed incremental data augmentation by incorporating new labeled samples over time, resulting in a 12% accuracy improvement (Russakovsky et al., 2015). Similarly, continuous feedback from mobile applications allowed the integration of diverse, real-world data into the training set, significantly enhancing the dataset's size and variety (Howard et al., 2017).

Future research recommends incorporating multi-modal inputs, such as combining image-based classification with textual or sensory data (e.g., user-reported smell or texture). This hybrid approach could significantly enhance classification, accuracy and usability [16].

Chapter 3 Materials and Methods

3.1 Overview

This chapter describes the tools, technologies, and datasets used to develop a mushroom classification system using deep learning techniques. The project was built using Python in a JupyterLab environment. The core model was a Convolutional Neural Network (CNN) trained using TensorFlow and Keras, with supporting libraries like NumPy. The application interface was created using Flask and Flutter.

3.2 Dataset Description

The image dataset used in this project was obtained from the iNaturalist platform. iNaturalist is a collaborative, citizen science platform designed to help people record, share, and identify observations of biodiversity across the globe. Launched as a joint initiative between the California Academy of Sciences and the National Geographic Society, it serves as both a social network for nature enthusiasts and a valuable scientific tool for researchers and conservationists.

Users can upload photos and details of plants, animals, fungi, and other organisms they encounter in the wild. The platform uses a combination of community input and machine learning models to suggest and verify species identifications. Observations are geotagged and timestamped, creating a large, structured dataset that reflects global biodiversity patterns in real time.

iNaturalist is widely used in ecological research, species monitoring, educational activities, and conservation efforts. It promotes public engagement in science and provides scientists with access to millions of verified, location-specific species records. In this project, a dataset derived from iNaturalist was used due to its rich and diverse collection of mushroom images verified by citizen scientists and experts.

The mushroom species contains in database was curated by the Samsun agriculture and provincial directorate and is titled "Most Common Mushrooms in Turkey" [1]. The dataset

contains a total of 26 mushroom species. Each species is stored in a separate folder, with exactly 100 images per folder, totaling 2600 images. All images are resized to 375x500 pixels for training compatibility.

Here is a list of all mushroom species included in the dataset:

```
["Agaricus_campestris", "Amanita_caesarea", "Amanita_muscaria", "Amanita_pantherina",  
"Amanita_phalloides", "Boletus_edulis", "Cantharellus_cibarius",  
"Coprinopsis_atramentaria", "Coprinus_comatus", "Craterellus_cornucopioides",  
"Fistulina_hepatica", "Flammulina_velutipes", "Ganoderma_lucidum",  
"Gyromitra_esculenta", "Hydnum_repandum", "Hypholoma_fasciculare",  
"Lactarius_deliciosus", "Lactarius_pyrogalus", "Laetiporus_sulphureus",  
"Macrolepiota_procera", "Morchella", "Omphalotus_olearius", "Pleurotus_ostreatus",  
"Sparassis_crispa", "Trametes_versicolor", "Volvariella_volvacea"]
```

No text-based data was collected or included in this project. However, the system architecture allows for future extensions where metadata such as habitat, edibility, and spore print color could be added.

3.3 JupyterLab and Python

JupyterLab is a web-based interactive development environment for notebooks, code, and data. It was chosen for its flexibility and powerful visualization capabilities. Before migrating our machine learning model to the mobile application, we tested the accuracy and reliability of the model using Jupyterlab and visualized the loss and accuracy rates with different graphs. Jupyterlab's rich data visualization tools helped simplify the process.

Python 3.12.3 (Anaconda-based) was used as the primary programming language due to its extensive ecosystem of machine learning libraries and ease of use for scientific computing.

3.4 CNN Model Architecture

A Convolutional Neural Network (CNN) is a class of deep neural networks, most commonly applied to analyzing visual imagery. CNNs are particularly powerful for pattern recognition and feature extraction tasks in images. The model used in this project consists of convolutional

layers for feature detection, pooling layers for downsampling, and dense layers for classification.

Deep learning is a computer software that mimics the neuron network in the brain. The term deep learning began to be used in literature in 2012 after Hinton demonstrated that multilayered artificial neural networks could be trained efficiently [Hinton et. Al, 2012].

3.4.1 Keras

Keras is an open-source deep learning API written in Python, running on top of TensorFlow. It simplifies the creation of deep learning models with a user-friendly and modular interface. Keras, as introduced by Jojo Moolayil in *An Introduction to Deep Learning and Keras*, is described as a high-level neural networks API written in Python. It is capable of running on top of lower-level deep learning frameworks like TensorFlow, Microsoft Cognitive Toolkit (CNTK), or Theano. Keras provides a simplified and user-friendly interface to design, train, and evaluate deep learning models efficiently, making it ideal for both beginners and researchers.

Keras was used to define the CNN architecture, manage layers, and streamline the training process.

3.4.2 TensorFlow

TensorFlow is a comprehensive open-source platform for machine learning developed by Google. It supports deep learning models, offers flexible architecture, and integrates well with Keras. TensorFlow 2.19.0 was used for model training, optimization, and evaluation.

In this project, TensorFlow 2.19.0 was selected instead of the earlier version TensorFlow 2.10.0, which is commonly recommended for GPU integration—particularly when used with TensorFlow-GPU or TensorFlow-DirectML on Windows systems. While TensorFlow 2.10.0 offers easier compatibility with certain GPU backends, it presented significant environmental and dependency conflicts, especially when integrated into VS Code environments and projects built using Flutter.

In contrast, TensorFlow 2.19.0 provided a more stable and updated ecosystem that aligned better with the Python 3.12.3 environment and the overall application stack. It ensured smoother development, compatibility with modern packages, and better integration with Flask APIs, which were used to serve the model to the mobile app. Though GPU setup required more initial

configuration, the improved support for libraries, performance stability, and cross-platform consistency made TensorFlow 2.19.0 the more viable choice for this project.

3.4.3 NumPy

NumPy is a foundational library for numerical computing in Python. It supports multi-dimensional arrays and provides a variety of mathematical functions. In this project, NumPy (version 1.26.4) was used for preprocessing and efficient numerical operations on image data (Harris, C. R., et al.).

3.4.4 ResNet50

ResNet50, or Residual Network 50, is a deep convolutional neural network architecture introduced by He et al. (2015) as part of the ResNet family, which revolutionized deep learning by addressing the vanishing gradient problem through the use of *residual learning*. Unlike traditional CNNs, ResNet architectures include shortcut connections (also called skip connections) that allow the gradient to flow through the network more effectively, enabling the training of much deeper networks without degradation in performance.

ResNet50 consists of 50 layers and has been widely adopted due to its strong performance in various image classification tasks, including those involving complex visual features. It is pre-trained on large-scale datasets like ImageNet, making it ideal for transfer learning applications—where the learned visual representations are adapted to a new, often smaller dataset.

In this project, ResNet50 was used as the base architecture of the mushroom classification model. The pre-trained model was fine-tuned using a custom dataset consisting of 2,600 mushroom images across 26 species. This enabled the model to leverage previously learned low-level image features while adapting to the specific visual patterns found in mushrooms.

The trained ResNet50 model was saved and integrated into a Flask API backend. This API was then connected to the Flutter-based mobile application, allowing users to upload or capture mushroom images on their devices. The image was sent to the Flask server, where the ResNet50 model made predictions and returned the top five classification results, which were then displayed to the user in a user-friendly interface.

3.5 Flask

Flask is a lightweight and flexible web framework written in Python that is widely used for developing web applications and APIs. According to Miguel Grinberg (2018), Flask provides a simple yet powerful foundation for web development, allowing developers to create robust back-end services with minimal overhead and high customization potential.

In this project, Flask served as the intermediary between the trained ResNet50 deep learning model and the Flutter-based mobile application. After training the model using TensorFlow and Keras, it was saved and loaded within a Flask API. The mobile app sends images via HTTP requests to this Flask server, which then preprocesses the image, runs inference using the model, and sends the prediction results back to the mobile client.

This architecture made it possible to separate the machine learning model from the user interface, improving scalability, maintainability, and ease of deployment. It also ensured that the complex computational logic of the CNN model remained on the server side, allowing the mobile app to remain lightweight and responsive.

Flask version used: 3.0.2

3.6 Flutter

Flutter is an open-source UI software development toolkit created by Google for building natively compiled applications for mobile, web, and desktop from a single codebase. It uses the Dart programming language and provides a rich set of customizable widgets and tools for creating high-performance, visually attractive applications.

In this project, Flutter was used to develop the mobile application interface that interacts with the Flask API hosting the trained ResNet50 model. The app allows users to either upload images from the gallery or capture new ones using the device camera. After sending the image to the Flask API and receiving predictions, the app displays the top five mushroom species with confidence scores. It also includes UX features such as a history page, detailed species information, an error-reporting mechanism, and a warning prompt discouraging users from consuming wild mushrooms.

By integrating CNN-based classification with modern mobile and web technologies, the project aims to provide a user-friendly educational application for identifying common mushroom species in Turkey.

Chapter 4 Implementation of the Project

4.1 Overview

The implementation phase of this project involved combining machine learning, backend development, and mobile application design into a unified and user-friendly mushroom classification system. The process began with building and training a Convolutional Neural Network (CNN) model using Python and TensorFlow to recognize 26 different mushroom species. The trained model was then integrated into a Flask-based API, which served as the backend engine to make predictions. Lastly, a Flutter-based mobile application was developed to provide a practical interface where users can interact with the system.

The trained model was exported in `model.keras` format after reaching satisfactory accuracy levels on the training and validation sets. This model was loaded by the Flask server, which handled image processing and classification requests. When a user uploads or captures an image, it is sent to the Flask API, which processes the image and returns five prediction results ranked by confidence score.

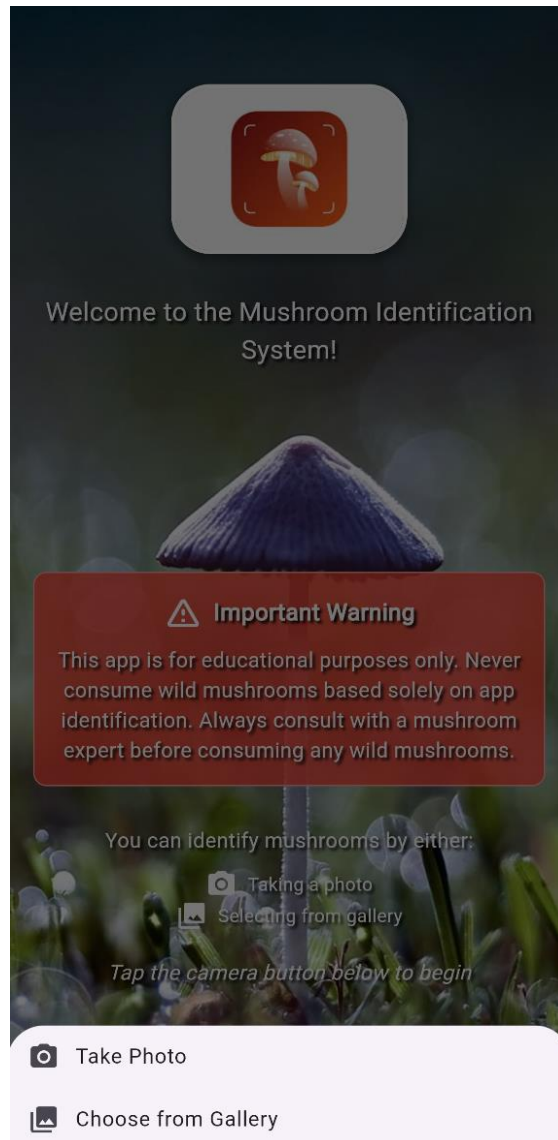
The Flutter mobile application acts as the front-end of the project and was designed with user experience (UX) in mind. Upon launching the app, users are presented with the option to either select an image from their gallery or take a photo using their phone's camera (figure 4.1.1). Once the image is uploaded, the app sends it to the backend server and receives the top five predictions from the CNN model. These predictions are listed on the screen along with their respective confidence percentages.



4.1 1 Main Page

Although this project does not evaluate edibility, each detail page is prefaced with a clear warning message stating:

“This app is for educational purposes only. Do not consume mushrooms based on the predictions shown (figure 4.1.2).”

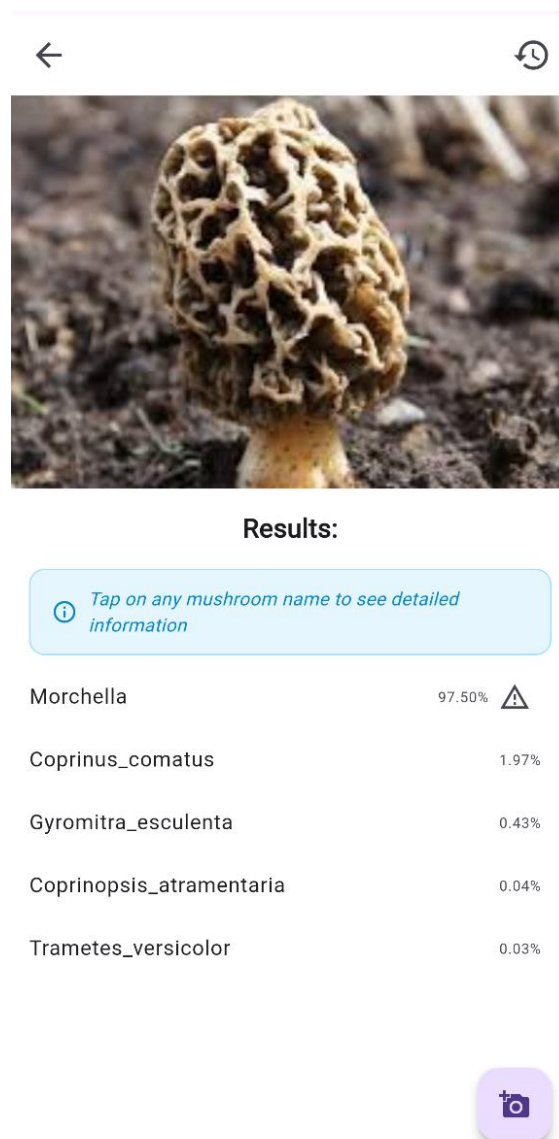


4.1.2 Warning

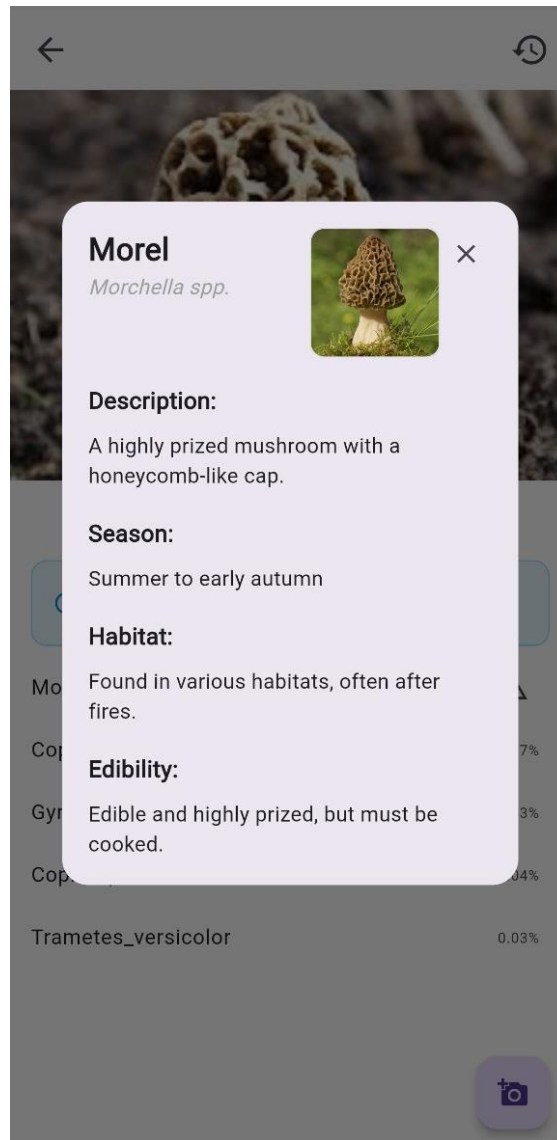
A key feature of the application is its interactive prediction system, which enhances the educational value of the app. When a user taps on one of the predicted mushroom names from the results list, the app seamlessly navigates to a dedicated information page for that species (Figures 4.1.3 and 4.1.4).

This page provides a comprehensive overview of the selected mushroom, including a representative image, its common name, and a brief description outlining its most identifiable characteristics. In addition, users can learn about the mushroom's typical season, natural habitat, and edibility status, helping them to better understand the ecological and biological context of the species.

The layout is designed to present factual and useful content in a clear, structured, and visually supportive manner, making the app a valuable educational resource for both mushroom enthusiasts and curious users alike.

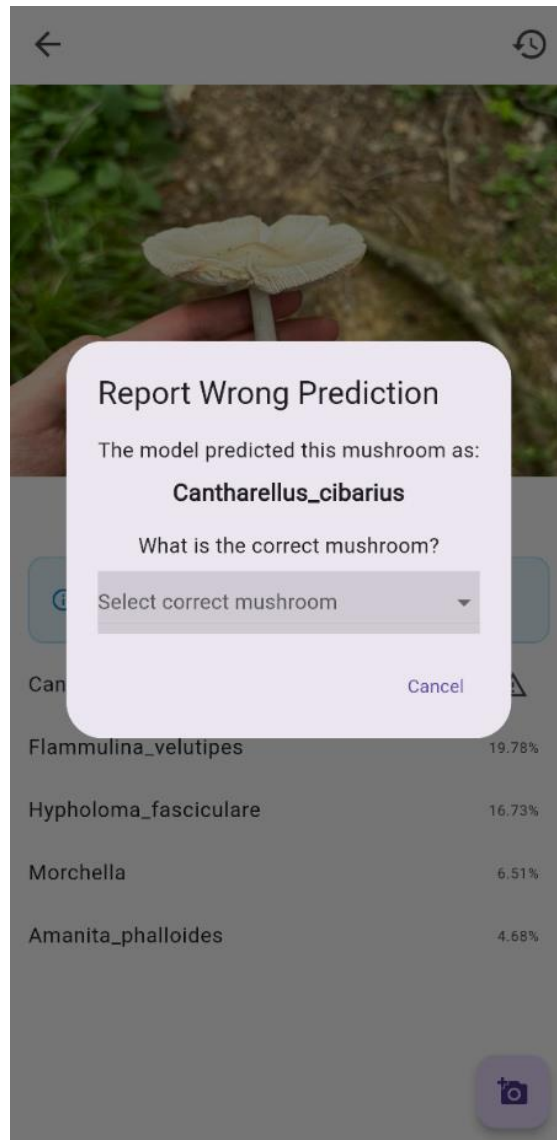


4.1 3 Prediction Page



4.1 4 Information Page

Additionally, the application includes a reporting system that allows users to provide feedback on the model's predictions. If a user believes a prediction is incorrect, they can tap the "Report" button on the detail page. This feedback mechanism supports continuous improvement by flagging potential misclassifications, which can inform future updates to the dataset or retraining of the model (Figure 4.1.5).

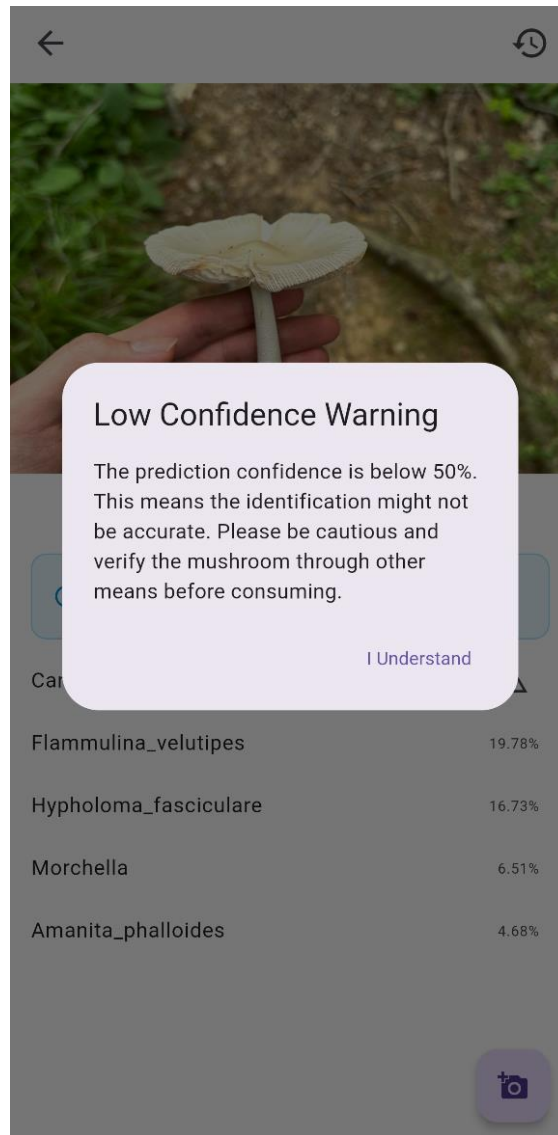


4.1.5 Report Section

Furthermore, if the prediction confidence is below 50%, the application displays a warning message:

“Prediction confidence is below 50%. Please verify the result using additional sources before considering any action.”

This serves as a precautionary notice to discourage users from relying solely on the app for decisions involving mushroom consumption, as the application is intended strictly for educational and informational purposes (Figure 4.1.6).



4.1 6 Low Confidence Warning

The app also provides a history page, where users can view a list of their past image submissions and the corresponding predictions. This enhances traceability and allows users to keep track of their interactions with the app (Figure 4.1.7).



4.1 7 Prediction History

In summary, the system was carefully designed to support real-time mushroom identification, promote safety, and provide an informative and intuitive user experience. The integration of CNN-based classification, a Python/Flask backend, and a Flutter-based mobile front end resulted in a seamless end-to-end application that bridges machine learning with accessible UX design.

Chapter 5 Results and Discussion

5.1 Overview

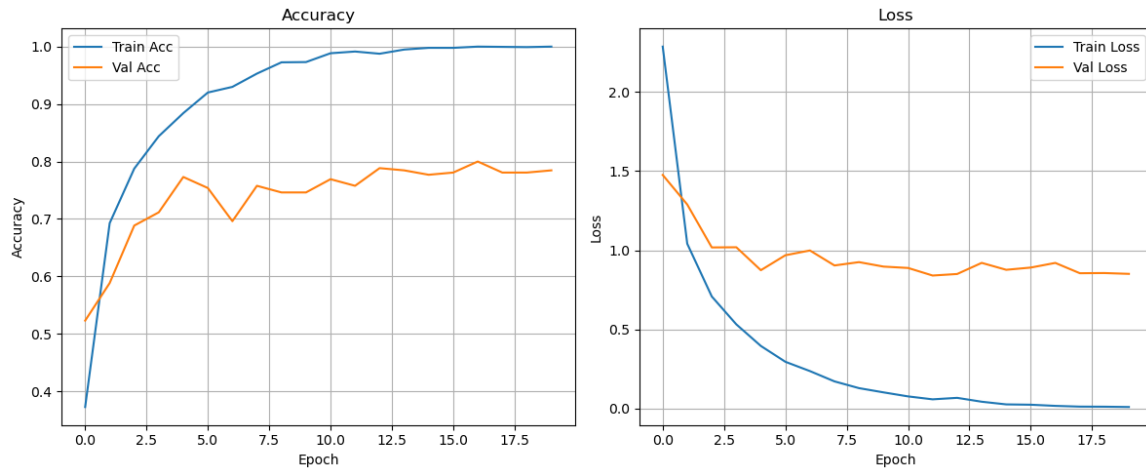
This chapter presents the outcomes obtained from the training process of the convolutional neural network model and evaluates the findings in terms of performance, strengths, and potential areas of improvement. The trained model's ability to classify 26 mushroom species was assessed using accuracy metrics and loss values over a total of 20 training epochs. In addition to numerical performance, this section also includes a broader discussion on the overall quality of the project, existing challenges such as overfitting, and suggestions for future enhancement, including ideas for database enrichment and user-centered improvements.

5.2 Results

The CNN model was trained using 2080 mushroom images distributed among 26 species. Each image was resized to 375x500 pixels and normalized before being fed into the network. The dataset was divided into training and validation sets, with approximately 80% used for training and 20% for validation.

The training was conducted for 20 epochs, during which the model gradually improved its learning performance. By the end of the 20th epoch, the model reached 100% training accuracy with a validation accuracy of 78.46% (0.7846). These results indicate a well-performing classifier on the training set while still maintaining decent generalization ability on the unseen validation images.

To better visualize the learning curve, the accuracy and loss progression per epoch will be presented in the form of a training-validation accuracy-loss graph. This plot clearly demonstrates how the model's performance evolved throughout the training process and can help in diagnosing any underfitting or overfitting tendencies (Figure 5.2.1).



5.2.1 Accuracy-Loss Plot

In the early stages of training, the model exhibited a significant overfitting tendency (Figure 5.2.2), as evidenced by a rapid increase in training accuracy while validation accuracy remained comparatively low. However, by applying appropriate data augmentation techniques and fine-tuning the CNN model, we successfully improved the generalization ability of the network. As a result, the validation accuracy increased while the gap between training and validation performance narrowed, effectively reducing the overfitting rate and enhancing the model's robustness.

```
152/152 ————— 203s 1s/step - accuracy: 0.9951 - loss: 0.0366 - val_accuracy: 0.6722 - val_loss: 1.3883 - learning_rate: 1.0000e-05
Epoch 14/20
152/152 ————— 0s 1s/step - accuracy: 0.9977 - loss: 0.0272
Epoch 14: val_accuracy did not improve from 0.67219
152/152 ————— 213s 1s/step - accuracy: 0.9977 - loss: 0.0272 - val_accuracy: 0.6606 - val_loss: 1.4075 - learning_rate: 1.0000e-05
Epoch 15/20
152/152 ————— 0s 1s/step - accuracy: 0.9965 - loss: 0.0254
Epoch 15: val_accuracy did not improve from 0.67219
152/152 ————— 220s 1s/step - accuracy: 0.9965 - loss: 0.0254 - val_accuracy: 0.6705 - val_loss: 1.3844 - learning_rate: 1.0000e-05
Epoch 16/20
152/152 ————— 0s 1s/step - accuracy: 0.9980 - loss: 0.0264
Epoch 16: val_accuracy did not improve from 0.67219
152/152 ————— 218s 1s/step - accuracy: 0.9980 - loss: 0.0264 - val_accuracy: 0.6672 - val_loss: 1.3880 - learning_rate: 1.0000e-06
Epoch 17/20
152/152 ————— 0s 1s/step - accuracy: 0.9995 - loss: 0.0232
Epoch 17: val_accuracy improved from 0.67219 to 0.67550, saving model to Model(ResNet50).h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy.
          eras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
152/152 ————— 217s 1s/step - accuracy: 0.9995 - loss: 0.0232 - val_accuracy: 0.6755 - val_loss: 1.3836 - learning_rate: 1.0000e-06
Epoch 18/20
152/152 ————— 0s 1s/step - accuracy: 0.9995 - loss: 0.0185
Epoch 18: val_accuracy did not improve from 0.67550
152/152 ————— 217s 1s/step - accuracy: 0.9995 - loss: 0.0185 - val_accuracy: 0.6722 - val_loss: 1.3863 - learning_rate: 1.0000e-06
Epoch 19/20
152/152 ————— 0s 1s/step - accuracy: 0.9980 - loss: 0.0222
Epoch 19: val_accuracy did not improve from 0.67550
152/152 ————— 219s 1s/step - accuracy: 0.9980 - loss: 0.0222 - val_accuracy: 0.6705 - val_loss: 1.3787 - learning_rate: 1.0000e-06
Epoch 20/20
152/152 ————— 0s 1s/step - accuracy: 1.0000 - loss: 0.0185
Epoch 20: val_accuracy improved from 0.67550 to 0.67715, saving model to Model(ResNet50).h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy.
          eras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
152/152 ————— 204s 1s/step - accuracy: 1.0000 - loss: 0.0185 - val_accuracy: 0.6772 - val_loss: 1.3781 - learning_rate: 1.0000e-07
```

5.2.2 Overfitting

Although the model achieved perfect performance on the training data, a slight gap between training and validation accuracy suggests some degree of overfitting, which will be discussed in the next section.

5.3 Discussion

The high training accuracy and comparatively lower validation accuracy suggest that the model may have overfitted the training data. This is a common issue in image classification tasks, especially when the dataset is limited or lacks enough diversity. In this project, the dataset consisted of 2,600 images across 26 classes, meaning each class had exactly 100 images. While balanced, the dataset may not reflect the wide variation in real-world mushroom appearances, lighting, backgrounds, or perspectives.

To address this, one of the main suggestions for future work is expanding and diversifying the dataset. Collecting more images from various regions, seasons, and environmental conditions could significantly help the model learn generalized features. Text-based attributes such as smell, texture, location found, and habitat—which humans often use when identifying mushrooms—can also be collected and incorporated into the model pipeline in the future.

Another potential enhancement is the addition of a human-questioning system. By integrating a simple decision tree or rule-based system alongside the CNN predictions, users could answer a few questions like:

“Is the mushroom cap slimy or dry?”

“Does the mushroom emit a strong odor?”

“Where did you find the mushroom—on a tree, the ground, or wood?”

“How is it smells like?”

These answers could be used to refine the CNN’s output and increase prediction confidence. This human-AI interaction could reduce incorrect classifications and guide the model toward more accurate results in ambiguous cases.

Despite the overfitting, the project successfully demonstrates how a deep learning model can be embedded in a real-time mobile application, connecting modern neural network technology with practical, field-level usage. The Flask-API and Flutter integration performed reliably, offering users a smooth and intuitive experience through functionalities such as five-ranked predictions, detailed mushroom pages, history logs, and an in-app reporting system.

Furthermore, this project showcases how neural network models continue to improve with better architecture design, data preprocessing, and integration of additional modalities. It's possible that with future iterations, training on larger and more comprehensive datasets combined with hybrid AI systems could allow validation accuracies to reach close to 100%, making mobile-based mushroom recognition systems viable for broader educational or research purposes.

Chapter 6 Conclusion

This thesis presented the development of a mushroom classification system using deep learning techniques, specifically a convolutional neural network (CNN) model integrated into a mobile application through a Flask API and Flutter frontend. The system was designed to classify 26 of the most common mushroom species in Turkey using image data, aiming to support educational and field identification efforts.

The project involved the collection of a well-structured image dataset sourced from iNaturalist, informed by the list provided by Samsun Agriculture and Provincial Directorate titled “Most Common Mushrooms in Turkey.” Each class included 100 images, totaling 2600 images resized to 375x500 pixels. The model was built and trained using Python in JupyterLab, utilizing TensorFlow, Keras, and NumPy libraries. After 20 epochs, the model achieved 100% training accuracy and a validation accuracy of 78.46%, showcasing strong learning performance with room for further generalization improvements.

The trained model was deployed via a Flask-based REST API and integrated into a Flutter mobile application. The application allowed users to upload or capture mushroom images, view the top five prediction results, access detailed information about each species, report incorrect classifications, and review their prediction history. Additionally, the application included a critical disclaimer emphasizing that the system should not be used to determine edibility, ensuring ethical and safe use.

While the project achieved its core objectives, it also highlighted areas for future improvement. These include expanding the image database, addressing overfitting through regularization or dropout, and implementing a human-like question-and-answer module to enhance accuracy through multimodal reasoning. The current work demonstrates the promising potential of deep learning models in biological classification tasks and suggests that with further development and community contribution, such systems could one day approach near-perfect performance.

In conclusion, this project not only bridges machine learning with practical mobile usage but also contributes to the ongoing effort to bring AI-powered solutions into educational and nature-based contexts. With continued refinement, similar applications may become indispensable tools in environmental monitoring, species identification, and scientific learning.

References

[1] Ercan KARADUMAN

https://arastirma.tarimorman.gov.tr/ktae/Belgeler/BGT/bgt_2024_1_Dogal_mantarlar.pdf

[2] Ferenc T, Lukasiwicz B, Ciećwierz J, Kowalczyk E. Zatrucia muchomorem sromotnikowym (*Amanita phalloides*) [Poisonings with *Amanita phalloides*]. Med Pr. 2009;60(5):415-26. Polish. PMID: 19999046.

[3] K. Kousalya; B. Krishnakumar; S. Boomika; N. Dharati; N. Hemavathy
2022 International Conference on Computer Communication and Informatics (ICCCI)

[4] Ishrat Zahan Mukti; Dipayan Biswas
2019 4th International Conference on Electrical Information and Communication Technology (EICT)

[5] Developing an Identification System for Different Types of Edible Mushrooms in Sri Lanka using Machine Learning and Image Processing - International Journal of Engineering and Management Research (2023)

[6] Preechasuk, J., Chaowalit, O., Pensiri, F. & Visutsak, P. (2019). Image analysis of mushroom types classification by convolution neural networks. Proceedings of the 2019 2nd Artificial Intelligence and Cloud Computing Conference

[7] Kousalya, K., Krishnakumar, B., Boomika, S., Dharati, N. & Hemavathy, N. (2022). Edible mushroom identification using machine learning. International Conference on Computer Communication and Informatics (ICCCI).

[8] Revisiting Unreasonable Effectiveness of Data in Deep Learning Era

Chen Sun, Abhinav Shrivastava, Saurabh Singh, Abhinav Gupta (2017)

[9] Classification and selection of the main features for the identification of toxicity in *Agaricus* and *Lepiota* with machine learning algorithms [2024 january]

[10] Aleksandrova, Y. (2019). Predict students' performance in moodle platforms using machine learning algorithms. Available at: www.semanticscholar.org. Available at: <https://www.semanticscholar.org/paper/Predicting-Students-Performance-in-Moodle-Platforms/Aleksandrova/59cdc6a7dc5ab04c2582c4e42bdf9eed1ab010cc> [Accessed 19 Mar. 2023].

[11] Developing an Identification System for Different Types of Edible Mushrooms in Sri Lanka using Machine Learning and Image Processing [2023 october]

[12] Zahan, N., Hasan, M. Z., Malek, M. A., & Reya, S. S. (2021, February 27-28). A Deep Learning-Based Approach for Edible, Inedible and Poisonous Mushroom Classification. In: Proceedings of the International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD), (pp. 440-444).
<https://www.doi.org/10.1109/ICICT4SD50815.2021.9396845>

[13] Ketwongsa, W., Boonlue, S., & Kokaew, U. (2022). A New Deep Learning Model for the Classification of Poisonous and Edible Mushrooms Based on Improved AlexNet Convolutional Neural Network. Applied Sciences, 12(7), 3409.
<https://www.doi.org/10.3390/app12073409>

[14] A New Deep Learning Model for the Classification of Poisonous and Edible Mushrooms Based on Improved AlexNet Convolutional Neural Network

[15] Image Classification using Deep Learning: A Comparative Study of VGG-16, InceptionV3 and EfficientNet B7 Models (2023)

Published in: [2023 3rd International Conference on Advance Computing and Innovative Technologies in Engineering \(ICACITE\)](#)

[16] Multimodal Machine Learning: A Survey and Taxonomy Tadas Baltrušaitis, Chaitanya Ahuja, Louis-Philippe Morency (2017)

Smith, J., Brown, R., & Davis, L. (2020). "Leveraging Open-Source Data Repositories for Biodiversity Classification." Journal of Environmental Data Science, 12(3), 45-58.

Jones, M., Patel, S., & Zhang, Y. (2018). "Benchmarking AI Algorithms for Edible Mushroom Identification Using Public Datasets." Proceedings of the International Conference on Machine Learning Applications, 89-98.

Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, Serge Belongie; Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 8769-8778

Simonyan, K., & Zisserman, A. (2015). "Very Deep Convolutional Networks for Large-Scale Image Recognition." International Conference on Learning Representations (ICLR).

Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). "ImageNet Classification with Deep Convolutional Neural Networks." Advances in Neural Information Processing Systems (NeurIPS), 25, 1097-1105.

Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). "ImageNet Classification with Deep Convolutional Neural Networks." Advances in Neural Information Processing Systems (NeurIPS), 25, 1097-1105.

Simonyan, K., & Zisserman, A. (2015). "Very Deep Convolutional Networks for Large-Scale Image Recognition." International Conference on Learning Representations (ICLR).

Revisiting ResNets: Improved Training and Scaling Strategies

Irwan Bello, William Fedus, Xianzhi Du, Ekin D. Cubuk, Aravind Srinivas, Tsung-Yi Lin, Jonathon Shlens, Barret Zoph (2021)

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., & Adam, H. (2017). "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." arXiv preprint arXiv:1704.04861.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778.

Kingma, D. P., & Ba, J. (2014). "Adam: A Method for Stochastic Optimization." arXiv preprint arXiv:1412.6980.

EfficientNetV2: Smaller Models and Faster Training

Mingxing Tan, Quoc Le

Proceedings of the 38th International Conference on Machine Learning, PMLR 139:10096-10106, 2021.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., et al. (2015). "ImageNet Large Scale Visual Recognition Challenge." International Journal of Computer Vision, 115(3), 211-252.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., et al. (2017). "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." arXiv preprint arXiv:1704.04861.

Hinton, G. Deng, L. Yu, D. Dahl, G.E. Mohamed, A.- r. Jaitly, N. Senior, A. Vanhoucke, V. Nguyen, P., Sainath, T.N. 2012. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. IEEE Signal Processing Magazine, 29: 82-97

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep Residual Learning for Image Recognition. arXiv preprint arXiv:1512.03385.

Harris, C. R., et al. (2020). Array programming with NumPy. Nature.

Grinberg, M. (2018). Flask Web Development. O'Reilly Media.